

Mini Transaction Ledger

Technical Documentation & System Architecture

Date: September 2026 · GitHub Repository:

<https://github.com/SaifullahMnsur-Again/Mini-Transaction-Ledger>

1 App Architecture & System Overview

The application follows **Domain-Driven Design (DDD)** and Clean Architecture principles, separated into three primary tiers:

- **Client Tier (React 18 + TypeScript + Vite):** Serves single-page application (SPA) views for Chart of Accounts, Journal Posting, and Account Statements. Enforces client-side validation (verifying total debits strictly equal total credits) and communicates with the backend via Axios.
- **Backend API Tier (.NET 10 Web API):** Enforces core financial invariants. `AccountsController` handles account management, `TransactionsController` manages atomic double-entry postings, and `LedgerEngineService` processes double-entry rules inside database transactions.
- **Persistence Tier (PostgreSQL 16 + EF Core 10):** Stores accounts, transactions, and split legs using fixed-point precision (`decimal(18, 4)`). Database migrations are version-controlled in `backend/Migrations`.

2 Key Code Components & Purpose

2.1 Backend (.NET 10)

- `LedgerDbContext.cs`: Configures Entity Framework Core entity mappings, foreign key relations, decimal column precision, and seed data.
- `Migrations/`: Version-controlled SQL migration files for reliable schema deployment.
- `AccountsController.cs`: Exposes REST API endpoints for account classification management and statement queries.
- `TransactionsController.cs`: Endpoints for posting atomic double-entry journal entries.
- `LedgerEngineService.cs`: Core financial engine enforcing $\sum \text{Debits} = \sum \text{Credits}$, normal balance rules, and running balance calculation.

2.2 Frontend (React 18)

- `ChartOfAccounts.tsx`: Interactive UI displaying accounts grouped by type with real-time balance totals.
- `JournalEntryForm.tsx`: Dynamic posting form with live equilibrium status indicator pills (`Dr == Cr`).
- `AccountStatement.tsx`: Renders chronological T-account ledgers with counter-party details and CSV export.

3 Internal API Workflow: Posting a Transaction

1. **Client Dispatch:** The React SPA validates form completeness and issues a POST `/api/v1/transactions` request.
2. **Invariant Verification:** `LedgerEngineService` checks that Total Debits equal Total Credits, amounts are greater than zero, and destination accounts exist.
3. **Database Transaction:** An explicit EF Core transaction starts (`BeginTransactionAsync`).
4. **Balance Calculation:** Account balances update based on normal balance conventions, and each split records a `RunningBalanceAfter` snapshot.
5. **Atomic Commit:** Changes persist via `SaveChangesAsync()`, the transaction commits, and a 201 `Created` status returns to the client.

4 Docker Setup & Containerization

- **ledger-db (PostgreSQL 16):** Containerized database with health checks (`pg_isready`) ensuring readiness before dependent services launch.
- **ledger-backend (.NET 10):** Multi-stage Dockerfile utilizing BuildKit cache mounts for fast NuGet restores and lightweight runtime output.
- **ledger-frontend (Node 20 / Nginx):** Multi-stage build serving optimized static assets.
- **ledger-net:** Isolated Docker bridge network facilitating internal service resolution by container name.

5 Concurrency, Data Integrity & Performance

- **Transaction Isolation:** Postings run inside explicit SQL transactions (`BeginTransactionAsync`) to ensure ACID compliance and prevent race conditions.
- **Indexing:** Indexes on (`AccountId`, `PostedAtUtc`) optimize statement queries from $O(N)$ table scans to $O(\log N)$ index lookups.
- **Fixed-Point Precision:** All monetary values enforce exact fixed-point `decimal(18, 4)` precision to prevent floating-point rounding errors.

6 Architectural Trade-offs & Future Enhancements

- **Calculated Running Balances:** Snapshots are computed at posting time for fast statement reads, though high-volume concurrent entries to a single account require database row locking.
- **Future Scaling:** Implementation of Event Sourcing/CQRS, multi-currency support, and period-end closure workflows.